

PreOne Enterprise

Test Strategy

Chapter 11 — Enterprise Preschool Operating System

Document Version: 1.0

Status: Test Strategy Freeze

Chapter Reference: 11 (Master PRD)

Date: 2026-07-14

Product: PreOne — Enterprise Preschool Operating System

Predecessor: Vision v1.0, BRC v1.0, Master PRD v1.0, DDD v1.0, ERD v3.0, API Contract Catalog v1.0, OpenAPI Spec v1.0, ADR-111 DevOps v1.0, ADR-113 Security v1.0

Successor: Test Plan, Test Case Catalog, Test Data Catalog, API Test Collection, E2E Automation Suite, Performance Test Suite, Security Test Checklist, CI/CD Quality Gate Report, Coverage Report

Scope: Backend (NestJS) + Web Portal (Next.js) + PreOne Hub (React Native) + APIs + Database + Infrastructure

Classification: Internal Engineering Reference

Prepared by: PreOne QA & Engineering Team

11.1 Testing Philosophy.....	1
11.1.1 Core Principles.....	1
11.1.2 Principle Enforcement in CI.....	2
11.1.3 Test Culture.....	2
11.2 Test Pyramid.....	2
11.2.1 Pyramid Visualization.....	2
11.2.2 Coverage Targets by Layer.....	4
11.2.3 Why the Pyramid Works.....	5
11.2.4 Anti-Patterns to Avoid.....	5
11.3 Unit Tests.....	5
11.3.1 Purpose.....	6
11.3.2 Tools by Layer.....	6
11.3.3 Coverage Rules.....	6
11.3.4 Example — StudentService.....	7
11.3.5 Mocking Strategy.....	7
11.3.6 Test Naming + Structure.....	7
11.4 Integration Tests.....	8
11.4.1 Purpose.....	8
11.4.2 Validations.....	8
11.4.3 Database Testing Pattern.....	9
11.4.4 Example — Admission Flow.....	9
11.4.5 Test Database Management.....	11
11.4.6 Multi-Tenant Isolation Tests.....	11
11.5 API Tests.....	12
11.5.1 Scope.....	12
11.5.2 Validations.....	12
11.5.3 Test Types.....	13

11.5.4 Example — POST /api/v1/students.....	13
11.5.5 Contract Drift Detection.....	13
11.5.6 Test Execution.....	14
11.5.7 API Test Collection.....	14
11.6 End-to-End (E2E) Tests.....	14
11.6.1 Critical Business Flows.....	14
11.6.2 Platforms.....	15
11.6.3 Example — Parent Fee Payment Flow.....	15
11.6.4 Test Data for E2E.....	18
11.6.5 Failure Handling.....	18
11.6.6 E2E Suite Maintenance.....	18
11.7 Performance Tests.....	19
11.7.1 Objectives.....	19
11.7.2 Test Types.....	19
11.7.3 Target SLAs.....	20
11.7.4 Load Test Architecture.....	20
11.7.5 Performance Regression Detection.....	20
11.7.6 Capacity Planning.....	21
11.8 Security Tests.....	21
11.8.1 Authentication Tests.....	21
11.8.2 Authorization Tests.....	21
11.8.3 OWASP Top 10 Coverage.....	22
11.8.4 Security Activities.....	22
11.8.5 Remediation SLAs.....	23
11.9 Test Data Strategy.....	23
11.9.1 Principles.....	23
11.9.2 Environment-wise Data Strategy.....	23
11.9.3 Standard Seed Dataset.....	24

11.9.4 Synthetic Data Generation.....	28
11.9.5 Masking Pipeline.....	28
11.9.6 Test Data Refresh Cadence.....	28
11.10 Test Environments.....	28
11.10.1 Environment Catalog.....	29
11.10.2 Promotion Gates.....	29
11.10.3 Environment Hygiene.....	29
11.10.4 Environment Monitoring.....	30
11.11 CI/CD Quality Gates.....	30
11.11.1 Gate Catalog.....	30
11.11.2 Enforcement Levels.....	30
11.11.3 Gate Bypass Policy.....	31
11.11.4 Gate Reporting.....	31
11.11.5 Gate Maintenance.....	31
12. Deliverables.....	32
12.1 Deliverable Lifecycle.....	32
12.2 Deliverable Storage.....	32
13. Cross References.....	32
13.1 Document Control.....	33
14. Glossary.....	33

11.1 Testing Philosophy

Testing is the engineering discipline that converts intent into verified behavior. In a multi-tenant SaaS serving hundreds of preschools across India, an untested code path is not a risk — it is a guarantee of a future incident involving children's data. PreOne's testing philosophy is therefore uncompromising: every business rule is tested, every bug fix

includes a regression test, and CI/CD blocks on every failure. This chapter defines the seven core principles that govern how PreOne engineers write, run, and maintain tests across the backend, web portal, mobile app, APIs, database, and infrastructure.

The principles below are not aspirational — they are enforced. CI/CD pipelines encode them as gates that cannot be bypassed by individual engineers. Code review checklists reference them. Onboarding training instills them. When a principle conflicts with shipping speed, the principle wins; a delayed release is recoverable, a leaked child record is not. Engineers who internalize these principles produce code that is testable by construction, with dependencies injectable, side effects isolated, and business logic separated from infrastructure.

11.1.1 Core Principles

Each principle below is accompanied by its concrete implementation in the PreOne stack. Engineers are expected to apply these principles in day-to-day design decisions — not only when writing tests, but also when designing features, refactoring code, or reviewing peers' work. A pull request that violates a principle will be blocked by the reviewer even if the immediate change appears harmless.

Principle	Description & Implementation
Test Early (Shift Left)	Testing begins at the requirements stage, not after code is written. Engineers write test plans alongside feature specs; QA reviews PRDs for testability. Defects caught in requirements cost 100x less than defects caught in production.
Automate Everything Possible	If a test is run more than once, it must be automated. Manual testing is reserved for exploratory, usability, and visual-verification tasks. Automation is treated as production code — reviewed, versioned, and maintained.
Test Every Business Rule	Every rule in BRC v1.0 (fee calculation, attendance policy, approval workflow, consent, retention) has at least one test per scenario. Business-rule tests are tagged in the test suite and reported separately on the QA dashboard.

Principle	Description & Implementation
Trace Every Test to PRD/BRC	Each test case carries a traceability tag linking it to a PRD requirement ID or BRC rule ID. Untested requirements are surfaced in coverage reports. Traceability matrix is auto-generated from test metadata.
Every Bug Must Have a Regression Test	Bug fixes cannot merge without a failing test that reproduces the bug and passes after the fix. This ensures the bug never reappears and documents the edge case for future engineers.
CI/CD Blocks on Failed Tests	A single failing test blocks the build. No exceptions, no overrides by individual engineers. Build breakers are reverted within 30 minutes. The team treats the main branch as always-deployable.
Security and Performance are Mandatory	Security scans (SAST, DAST, dependency) and performance benchmarks (latency, throughput) run on every PR. Failures block merge just like unit-test failures. Security/perf are not afterthoughts — they are first-class test types.

11.1.2 Principle Enforcement in CI

Several principles have automated enforcement in CI: (1) Automate Everything Possible — a PR that adds a manual test step to a release checklist is flagged for review. (2) Every Bug Must Have a Regression Test — bug-fix PRs without a failing test that reproduces the bug are blocked. (3) CI/CD Blocks on Failed Tests — the merge button is disabled until all gates pass; VP-level approval is required to override. (4) Security and Performance are Mandatory — SAST/DAST/dependency scans and k6 benchmarks are first-class gates, not afterthoughts. Principles without automated checks are enforced via mandatory code review + release sign-off.

11.1.3 Test Culture

Beyond tools and gates, PreOne cultivates a test-first engineering culture. New engineers complete a 'test dojo' during onboarding — a 2-day workshop covering the test pyramid,

mocking patterns, fixture design, and TDD. Engineering brown-bag sessions monthly highlight interesting test cases and testing anti-patterns. The QA team is embedded in feature squads, not siloed — a QA engineer reviews every feature spec at design time and writes test plans before code is written. Bugs are not blamed on individuals; they are treated as process gaps and trigger a '5 Whys' analysis.

11.2 Test Pyramid

The test pyramid is PreOne's foundational model for test-suite composition. The pyramid shape reflects volume: many fast unit tests at the base, fewer slower integration and API tests in the middle, very few slowest E2E tests at the top. This composition optimizes for two competing goals — fast feedback (favoring unit tests) and high confidence (favoring E2E tests). A pyramid that inverts (many E2E, few unit) is slow, brittle, and expensive; a pyramid that omits the top (no E2E) is fast but misses integration defects. PreOne's target shape is documented below and tracked via test-count metrics on the QA dashboard.

11.2.1 Pyramid Visualization

The diagram below illustrates the test pyramid. The arrow (▲) indicates that each layer builds on the confidence of the layer below. Volume decreases as you move up; speed decreases; cost increases; confidence in real-world behavior increases. The right composition is approximately 70% unit, 20% integration/API, 10% E2E.

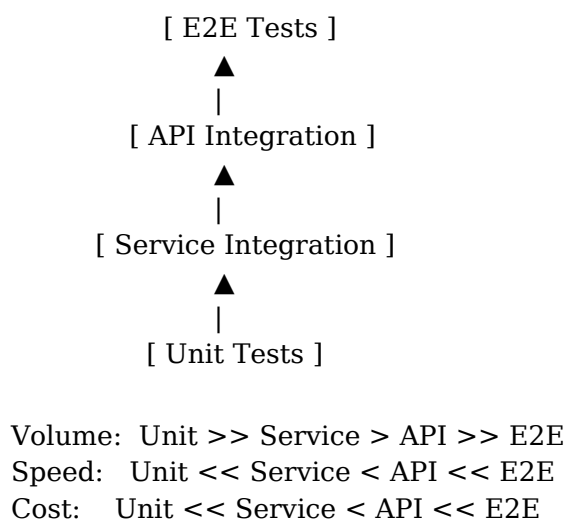


Figure 11.1 — Test pyramid: volume vs. speed vs. cost tradeoff across four layers

11.2.2 Coverage Targets by Layer

The table below enumerates each pyramid layer, its coverage target, a description, and the approximate test count in the PreOne suite. Coverage is measured as line + branch coverage via Jest. The targets are minimums — individual teams may set higher targets for critical modules. The QA dashboard tracks actual vs. target weekly; shortfalls are triaged in sprint planning.

Layer	Coverage Target	Description	Approx Count
Unit Tests	80–90%	Fast, isolated, deterministic. Validate individual functions, services, validators. Run on every save + every PR. Sub-second execution.	~8,500 tests
Service Integration	60–70%	Validate interaction between internal layers (controller-service-repository) and bounded contexts. Use real DB transaction rollback. Run on every PR.	~1,200 tests
API Integration	100% Critical APIs	Validate REST endpoints end-to-end through HTTP layer. Test auth, validation, response schema, errors, pagination, idempotency. Run on every PR + nightly.	~1,800 tests
E2E Tests	100% Critical User Flows	Validate complete user journeys across web + mobile + API. Slow, expensive — run nightly + pre-release. 10 critical flows covered.	~120 tests

Layer	Coverage Target	Description	Approx Count
Performance Tests	All critical modules	Load, stress, spike, endurance, scalability. Run nightly against staging. SLA-gated.	~45 scenarios
Security Tests	Entire platform	SAST + DAST + dependency + secret + image scan in CI. Pentest quarterly. OWASP Top 10 coverage.	Continuous + quarterly

11.2.3 Why the Pyramid Works

The pyramid works because each layer catches different defect classes at different costs. Unit tests catch logic bugs in milliseconds, allowing tight test-code-test loops during development. Integration tests catch wiring bugs (controller-service-repository miscommunication, schema drift, transaction bugs) in seconds. API tests catch contract bugs (validation, auth, response schema) in seconds. E2E tests catch flow bugs (multi-step user journeys, cross-system interactions) in minutes. By distributing tests across layers, PreOne catches each bug at the cheapest layer that can detect it — a logic bug caught by E2E is 100x more expensive to triage than the same bug caught by a unit test.

11.2.4 Anti-Patterns to Avoid

Three anti-patterns are explicitly forbidden in PreOne's test suite. (1) Ice-cream cone — many manual/E2E tests, few unit tests. Slow, brittle, expensive to maintain. (2) Hourglass — many unit tests, many E2E tests, few integration tests. Misses wiring bugs that E2E catches too late. (3) Cone inverted on security — security tested only via pentest, not via SAST/DAST in CI. Catches vulnerabilities too late, after code is shipped. Each anti-pattern is detected via test-count metrics on the QA dashboard and corrected in sprint planning.

11.3 Unit Tests

Unit tests validate individual units of code in isolation — a single function, method, or class. They are the foundation of the test pyramid: fast (sub-second), deterministic (no I/O), and

numerous (thousands). PreOne requires 85% overall unit-test coverage, with higher targets for domain and utility layers. Unit tests catch logic bugs at the cheapest point in the development cycle, provide living documentation of expected behavior, and enable safe refactoring — a developer who breaks behavior sees a unit test fail within seconds, not a QA engineer file a bug days later.

11.3.1 Purpose

Unit tests validate the smallest testable units of the codebase. In PreOne, a unit is typically a domain service method, an application service use case, a utility function, a DTO validator, a mapper, or a helper. Units are tested in isolation — all external dependencies (database, Redis, payment gateway, SMS, email, S3) are mocked. This isolation makes unit tests fast, deterministic, and focused on logic rather than infrastructure.

The unit types validated by PreOne's test suite include: domain services (business logic), application services (use-case orchestration), utility functions (pure helpers like date + money formatting), validators (DTO + custom business validators), mappers (entity-to-DTO and reverse), helpers (small reusable functions), and tools (CLI utilities, migration scripts). Each unit type has a corresponding test pattern documented in the engineering wiki.

11.3.2 Tools by Layer

Different layers use different unit-testing tools, optimized for the language + framework of that layer. The table below lists the tool for each layer. Jest is the primary tool across the stack because of its zero-config setup, snapshot testing, and parallel execution. Vitest is used selectively on the web frontend for Vite-native speed.

Layer	Tool	Description
Backend	Jest	NestJS services, controllers (logic only), domain entities, value objects, DTOs, validators, mappers. Configured with ts-jest. Coverage via Jest built-in.

Layer	Tool	Description
Frontend	Vitest / Jest	React components (Next.js), hooks, stores (Zustand/Redux), utility functions, form validators. Vitest preferred for Vite speed; Jest fallback for legacy modules.
Mobile	Jest	React Native components, hooks, navigation logic, offline-queue logic, sync reconcilers. Detox for E2E (separate suite).
Mocking	Jest Mocks	All external dependencies (DB, Redis, payment gateway, SMS, email, S3) mocked. No real I/O in unit tests. Mocks versioned alongside tests.

11.3.3 Coverage Rules

Coverage targets are tiered by layer criticality. The most critical layers (domain logic, utilities, critical business logic) have near-100% targets because bugs there propagate widely and are hard to detect via higher-level tests. Less critical layers (infrastructure adapters) have lower targets because they are better tested via integration tests. Coverage is measured per PR; a coverage drop blocks merge.

Scope	Target	Detail
Overall minimum	85%	Calculated via Jest coverage (lines, branches, functions, statements). Reported on every PR; build fails below threshold.

Scope	Target	Detail
Domain Layer	95%	Bounded contexts (Identity, Academic, Finance, HR, Inventory, Communication, Transport, etc.). Highest coverage because domain logic is the business differentiator.
Utility Layer	100%	Pure functions: date utils, money formatter, validators, mappers. Zero tolerance for uncovered utility — they are reused everywhere and bugs propagate widely.
Critical Business Logic	100%	Fee calculation, attendance aggregation, payroll computation, GPA calculation, consent enforcement. Each business rule has tagged tests.
Application Services	85%	Use-case orchestration layer. Mocked repositories; tests focus on orchestration logic + event publishing.
Infrastructure Layer	60%	Adapters for external systems. Lower target because much is integration-test territory. Tested with mocks; integration tests cover real behavior.

11.3.4 Example — StudentService

The table below illustrates the unit-test suite for StudentService, a typical domain service. Each test validates one behavior in isolation; mocks are used for the repository, event bus, and audit logger. The tests cover positive, negative, boundary, authorization, and integration concerns. Engineers writing a new service should follow this pattern — a service without an equivalent test suite cannot merge.

Test	Type	Description
Create Student — valid payload	Positive	Standard admission with all required fields. Asserts: student record created with status ACTIVE, admission number generated, audit log written, domain event StudentCreated emitted.
Update Student — partial update	Positive	PATCH with only changed fields. Asserts: only changed fields updated, updatedAt bumped, audit log captures old + new values.
Validate Age — within range	Boundary	DOB exactly 3y 0m 0d (min) and 6y 0m 0d (max) for preschool program. Asserts: accepted. Outside range — rejected with ELIGIBILITY_ERROR.
Duplicate Admission Number	Negative	Two students with same admission number in same branch. Asserts: second call throws ConflictError, no record written, audit logs the attempt.
Soft Delete — with dependencies	Positive	Delete student with active attendance + fee records. Asserts: student marked DELETED (not physically removed), related records preserved, future API calls exclude deleted student.
Soft Delete — permission denied	Authorization	Teacher attempts delete (only Principal+ allowed). Asserts: 403 Forbidden, audit logs unauthorized attempt.

Test	Type	Description
Multi-tenant isolation	Integration	Tenant A query must not return Tenant B students. Asserts: Prisma middleware enforces tenantId filter; cross-tenant access throws.

11.3.5 Mocking Strategy

Unit tests mock all external dependencies to ensure isolation and speed. PreOne uses Jest's built-in mocking (`jest.mock` + `jest.fn`) for all mocks. Mocks are versioned alongside tests and reviewed in PR. The mocking strategy follows three rules: (1) Mock at the boundary — mock the repository interface, not Prisma itself, so tests are not coupled to ORM internals. (2) Mock minimally — mock only what is needed for the test; over-mocking hides integration bugs. (3) Mock realistically — mock return values match real shapes; use fixture builders to keep mocks in sync with schemas.

11.3.6 Test Naming + Structure

Test names follow the pattern 'should <expected behavior> when <condition>'. Example: 'should return 404 when student not found'. This naming makes test failures self-documenting — the failing test name describes the bug. Test structure follows AAA (Arrange-Act-Assert): setup data, call the unit, assert the result. Each test asserts one behavior; multi-behavior tests are split. Setup is shared via `beforeEach` blocks; teardown via `afterEach`. Test files live alongside source files (`student.service.ts` → `student.service.spec.ts`).

11.4 Integration Tests

Integration tests verify that multiple units work correctly together. Where unit tests validate a single service in isolation, integration tests validate the wiring between layers: controller to service, service to repository, repository to database, event handlers, Redis, queue workers. Integration tests catch defects that unit tests cannot — misconfigured dependency injection, schema drift, transaction bugs, event-ordering issues, multi-tenant leakage. PreOne runs integration tests on every PR against an ephemeral Docker database, ensuring that wiring bugs are caught before merge, not in staging.

11.4.1 Purpose

Integration tests validate the interaction between the units validated by unit tests. The scope of integration tests at PreOne includes the boundaries listed below. Each boundary has its own test pattern + tooling, documented in the engineering wiki. Integration tests use real instances of dependencies (real Postgres, real Redis, real event bus) rather than mocks, because the goal is to catch integration defects.

Scope	Description
Controllers ↔ Services	Validate NestJS DI wiring, guard execution order, interceptor chain, exception filter. Mocked repositories; real service instances.
Services ↔ Repositories	Validate service-to-repository contract. Real Prisma client against test database (transaction-rollback pattern). Catches query bugs, mapping bugs, transaction bugs.
Repository ↔ Database	Validate Prisma schema, indexes, constraints, cascade behavior, soft-delete filters. Real Postgres instance in Docker; transaction rollback after each test.
Event Handlers	Validate domain event subscription, ordering, idempotency, retry. Real event bus (in-memory for tests); handlers exercised with real + simulated events.
Redis	Validate cache reads/writes, TTL behavior, pub/sub for real-time, BullMQ queue operations. Real Redis in Docker; flushdb between tests.
Queue Workers	Validate BullMQ job lifecycle: enqueue, process, retry on failure, dead-letter on max-attempts. Real Redis; jobs asserted to complete or fail predictably.

11.4.2 Validations

Integration tests validate behaviors that emerge from the interaction of multiple components. The validations below are mandatory coverage for every integration-test suite. A suite missing any validation is flagged in code review.

Validation	Description
Database Transactions	Multi-step operations (e.g., create invoice + update student balance + emit event) must succeed atomically. Rollback on any failure. Tests verify no partial state.
Rollbacks	Simulate failure at each step of a multi-step operation. Asserts: full rollback, no orphan records, audit log records the failed attempt.
Domain Events	Events published in transaction must be delivered exactly once. Tests verify event handler invocation, idempotency key, retry behavior, dead-letter on permanent failure.
Multi-Tenant Isolation	Every query in integration tests runs against a test database seeded with 2+ tenants. Tests assert no cross-tenant data leakage at any layer.
RBAC Enforcement	Each integration test runs as a specific role. Tests assert: allowed operations succeed; forbidden operations return 403; audit logs the decision.
Concurrency	Tests for optimistic-locking (version field), pessimistic locking (SELECT FOR UPDATE), and race conditions (parallel requests to same resource).

11.4.3 Database Testing Pattern

Integration tests that touch the database use the transaction-rollback pattern. Before each test, a transaction is begun; the test runs inside that transaction; after the test, the transaction is rolled back, leaving the database clean for the next test. This pattern is fast (no re-seed between tests) and isolated (no test pollution). Tests that need a clean schema (e.g., migration tests) use the test-container pattern — a fresh Postgres container per suite, torn down after.

11.4.4 Example — Admission Flow

The admission flow below is a canonical integration-test scenario. Each step involves multiple services + repositories + events. The integration test validates that the entire flow completes atomically — if any step fails, the entire flow rolls back, no partial state is left in the database, and the audit trail records the failed attempt. This test catches wiring bugs, transaction bugs, and event-ordering bugs that unit tests would miss.

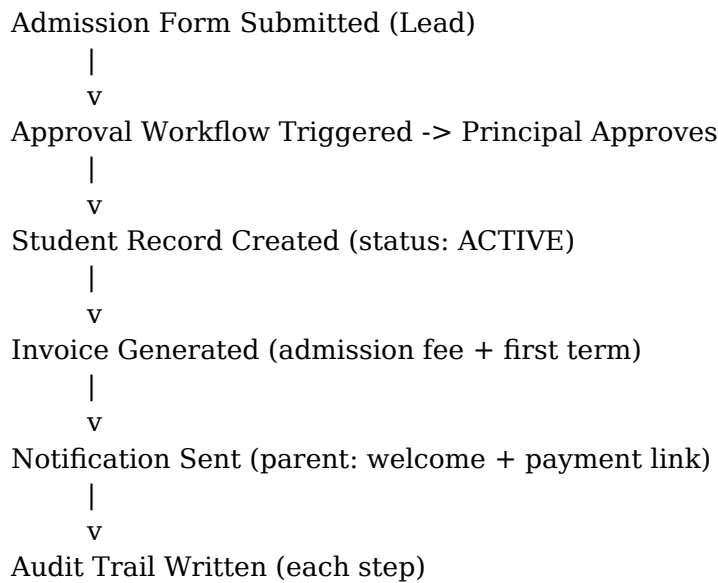


Figure 11.2 — Admission flow integration test: 5-step atomic transaction with event publishing + audit trail

11.4.5 Test Database Management

Integration tests use dedicated test databases, never production or shared dev databases. In CI, a fresh Postgres container is spun up per build; migrations applied; seed scripts run; tests execute; container torn down. In local dev, Docker Compose provides a persistent test database on port 5433, separate from the dev database on 5432. Tests are tagged (@integration) so they can be run independently of unit tests during development. The full integration suite runs in under 5 minutes — slow enough to catch real bugs, fast enough to run on every PR.

11.4.6 Multi-Tenant Isolation Tests

A specialized category of integration tests validates multi-tenant isolation. These tests seed the database with 2+ tenants, then attempt cross-tenant access at every layer: API, service, repository, raw SQL. Any cross-tenant access (read or write) fails the test. These tests run on every PR and are tagged @multi-tenant. They are the primary defense against data-leakage bugs that would be catastrophic for a multi-tenant SaaS.

11.5 API Tests

API tests validate the REST API surface end-to-end through the HTTP layer. Every API defined in the API Contract Catalog has corresponding API tests. Unlike unit tests (which validate service logic in isolation) and integration tests (which validate internal wiring), API tests validate the contract as seen by external clients — authentication, authorization, request validation, response schema, error handling, pagination, filtering, sorting, rate limiting, and idempotency. API tests are the primary defense against contract drift — a mismatch between the OpenAPI spec and the actual implementation fails the build.

11.5.1 Scope

API tests cover every REST API defined in the API Contract Catalog (v1.0) — approximately 530 endpoints across 14 domains. Each endpoint has a test suite covering positive, negative, boundary, authorization, validation, and performance test types. The suite is auto-generated in part from the OpenAPI spec — schema validation tests are generated; behavioral tests are hand-written. Test coverage is tracked per endpoint on the QA dashboard; an endpoint without tests is flagged.

11.5.2 Validations

Each API test validates the items below. The validations are layered — a request must pass all validations to return a successful response. Tests assert each validation independently so that a bug in one validation does not mask a bug in another.

Validation	Description
Authentication	Missing JWT → 401. Invalid JWT (bad signature, expired, wrong audience) → 401. Valid JWT → proceeds to authorization.
Authorization	Valid JWT but insufficient role/permission → 403. Each route tested with all role permutations; expected allow/deny matrix encoded as test data.
Request Validation	class-validator DTO rules enforced. Missing required field → 400. Wrong type → 400. Out-of-range value → 400. Whitelist mode — unknown fields rejected.

Validation	Description
Response Schema	Response matches OpenAPI spec. class-transformer @Exclude strips sensitive fields. Schema validated via jest-json-schema. Drift between code + spec fails the test.
Error Handling	Business errors return correct HTTP status + error code + message. Unhandled errors return 500 with generic message (no stack trace leaked). Errors logged with request ID.
Pagination	page + limit query params. Response includes items, total, page, limit, totalPages. Edge cases: page beyond range, limit=0 rejected, limit > max rejected.
Filtering	Filter query params (e.g., ?status=ACTIVE&branchId=X). Combined filters AND-ed. Invalid filter value → 400. Cross-tenant filter → 403.
Sorting	sortBy + sortOrder params. Allowed sort fields whitelisted per endpoint. Invalid sort field → 400. Default sort applied when not specified.
Rate Limiting	Exceeding per-IP / per-token limit → 429 with Retry-After header. Limit reset after window. Burst allowance enforced.
Idempotency	POST with Idempotency-Key → cached response on replay. Different payload with same key → 409 Conflict. Idempotency window 24h.

11.5.3 Test Types

Every API endpoint is tested with all six test types below. The Required column indicates mandatory coverage; an endpoint missing any required test type is flagged in code review. Performance tests are required but do not block PR merge — they are tracked for regression detection and reviewed at release time.

Test Type	Required	Description
Positive	✓ Required	Valid payload, valid token, valid role. Asserts: 2xx response, correct body, correct side effects (DB, events, audit).
Negative	✓ Required	Invalid payload, invalid token, expired token, wrong content-type. Asserts: 4xx response, correct error code, no side effects.
Boundary	✓ Required	Edge values: min/max length strings, min/max numeric, empty arrays, null vs undefined, date boundaries (leap year, timezone).
Authorization	✓ Required	Each role tested against each endpoint. Allow/deny matrix encoded in test data. Asserts: 200 for allowed, 403 for denied, audit logs decision.
Validation	✓ Required	Every DTO rule tested: required fields, type checks, format (email, phone, UUID), range, enum, custom validators.
Performance	✓ Required	Response time under 300ms for 95th percentile on standard payload. Tested in CI against staging. Failures logged but do not block PR (perf regression tracked separately).

11.5.4 Example — POST /api/v1/students

The table below illustrates the API test suite for the POST /api/v1/students endpoint — a representative endpoint with authentication, authorization, validation, boundary,

negative, and idempotency concerns. Each test makes a real HTTP request to an ephemeral API instance and asserts the response. Engineers adding a new endpoint should follow this pattern — a new endpoint without an equivalent test suite cannot merge.

Test	Type	Expected Result
POST /api/v1/students — valid payload (Principal role)	Positive	201 Created, student object with generated admissionNumber, audit log written
POST /api/v1/students — invalid DOB (future date)	Validation	400 Bad Request, code DOB_FUTURE_DATE, no record created
POST /api/v1/students — missing JWT	Authentication	401 Unauthorized, code TOKEN_MISSING
POST /api/v1/students — Teacher role (forbidden)	Authorization	403 Forbidden, code PERMISSION_DENIED, audit logged
POST /api/v1/students — duplicate admission number	Negative	409 Conflict, code DUPLICATE_ADMISSION_NUMBER
POST /api/v1/students — name field 501 chars (limit 500)	Boundary	400 Bad Request, code FIELD_TOO_LONG
POST /api/v1/students — cross-tenant branchId	Authorization	403 Forbidden, code BRANCH_NOT_IN_TENANT
POST /api/v1/students — idempotency replay	Idempotency	201 Created on first call; cached 201 on second call with same Idempotency-Key
POST /api/v1/students — response time	Performance	< 300ms p95 on staging with 100 concurrent users

11.5.5 Contract Drift Detection

API tests double as contract-drift detectors. Each test validates the response against the OpenAPI spec via jest-json-schema. If the implementation returns a field not in the spec, or omits a required field, or returns the wrong type, the test fails. This catches drift in both

directions — implementation diverging from spec, and spec diverging from implementation. The OpenAPI spec is the source of truth; tests fail until either the code or the spec is updated.

11.5.6 Test Execution

API tests run against an ephemeral API instance spun up per test suite in CI. The instance uses a real (containerized) database + Redis; external services (payment gateway, SMS, email) are mocked via WireMock or similar. Tests are parallelized across workers for speed; the full suite of ~1,800 tests runs in under 8 minutes. On failure, the test logs include the request, response, and server logs — sufficient for an engineer to reproduce and fix without re-running.

11.5.7 API Test Collection

Beyond the automated test suite, PreOne maintains a Postman collection mirroring the API tests. The collection serves two purposes: (1) manual exploratory testing by QA engineers, and (2) customer-facing API documentation with runnable examples. The collection is auto-generated from the OpenAPI spec; manual edits are forbidden. The collection is published to the customer developer portal and to internal teams via Postman workspaces.

11.6 End-to-End (E2E) Tests

End-to-end tests validate complete user journeys across the full stack — from the user interface (web or mobile) through the API, business logic, database, and external integrations. E2E tests are the slowest and most expensive layer of the pyramid, but they provide the highest confidence that the system works as users experience it. PreOne runs E2E tests nightly against staging and on-demand before every release. The suite covers ten critical business flows that span all 14 domains and exercise all integration points.

E2E tests are not a replacement for unit or integration tests — they are a complement. A passing E2E suite does not mean the system is bug-free; it means the critical paths work. Conversely, a failing E2E test signals a real user-impacting defect, even if unit + integration tests pass. Engineers investigating an E2E failure should treat it as a P1 bug until root-caused.

11.6.1 Critical Business Flows

The ten flows below constitute the PreOne E2E suite. Each flow exercises multiple domains, multiple integration points, and multiple user roles. The flows are selected based on business criticality — a failure in any of these flows would directly impact a preschool's daily operations or a parent's ability to engage with their child's education. Adding a new flow requires QA Lead + Product Manager approval.

Flow	Description
Lead → Admission	Sales/Principal creates a lead, follows up, converts to admission application, parent submits documents, principal approves, student record created. 9 steps; covers CRM + Admission + Identity.
Admission → Student	Approved admission triggers student record creation, class assignment, ID card generation, parent account creation, welcome notification. 7 steps; covers Admission + Academic + Identity + Notification.
Student Attendance	Teacher marks attendance in mobile app; absentees trigger parent notification; principal reviews daily summary; weekly aggregate pushes to parent dashboard. 6 steps; covers Academic + Notification + Reporting.
Daily Observation	Teacher records daily observation (activity, mood, meals, sleep) for a child; parent receives notification; parent can comment; weekly digest emailed. 5 steps; covers Academic + Communication + Notification.
Fee Payment	Parent views outstanding fees, initiates payment via UPI/card, gateway webhook updates invoice, receipt generated + emailed, student balance updated. 8 steps; covers Finance + Payment + Notification + Document.

Flow	Description
Parent Notification	School broadcasts announcement; parent receives push + in-app + email; parent acknowledges; school sees read-receipt dashboard. 5 steps; covers Communication + Notification + Analytics.
Inventory Purchase	Vendor creates PO; principal approves; goods received → stock updated; invoice processed; payment scheduled. 7 steps; covers Inventory + Finance + Approval.
Employee Payroll	HR runs monthly payroll; salary slips generated; finance approves payout; bank file generated; payslip emailed to staff. 6 steps; covers HR + Finance + Approval + Notification.
Transport Tracking	Driver starts trip; GPS pings every 30s; parent sees live ETA; pickup/drop OTP verified; trip closed; exceptions alerted. 8 steps; covers Transport + Notification + Real-time.
Report Generation	Teacher generates term report card; principal reviews; parent receives PDF via app + email; digital signature captured; archived for 5 years. 7 steps; covers Academic + Document + Notification + Compliance.

11.6.2 Platforms

E2E tests run on three platforms: Web Portal (Playwright), PreOne Hub mobile app (Detox), and API (Supertest). The web and mobile suites test the actual UI as users experience it; the API suite tests multi-endpoint flows where the UI is incidental (e.g., webhook processing, batch jobs). Each platform has its own tooling, optimized for that platform's execution model.

Platform	Tool	Description
Web Portal	Playwright	Cross-browser (Chromium, Firefox, WebKit). Cross-device viewport. Records video + trace on failure. Page Object Model architecture. Runs against staging nightly.
PreOne Hub (Mobile)	Detox	Gray-box E2E for React Native. Runs on Android emulator + iOS simulator in CI. Device farm for physical-device testing pre-release. Records video on failure.
API	Supertest	HTTP-level E2E for API flows that span multiple endpoints. Faster than UI E2E; used for flows where UI is incidental (e.g., webhook processing, batch jobs).

11.6.3 Example — Parent Fee Payment Flow

The flow below illustrates a typical E2E test: a parent logs in, views their child's outstanding fees, pays via UPI, downloads the receipt, and logs out. The test exercises: authentication (OTP), dashboard rendering, invoice display, payment gateway integration, webhook processing, PDF generation, email notification, and session management. A failure at any step fails the test — engineers receive a video recording + trace for diagnosis.

```

Parent Login (OTP via SMS)
|
v
View Child Dashboard -> Outstanding Fees Section
|
v
Click 'Pay Now' -> Invoice Details -> UPI/Card Selection
|
v
Payment Gateway (Razorpay) -> OTP/3DS Authentication
|
v
Gateway Webhook -> Invoice Updated (PAID) -> Receipt Generated

```

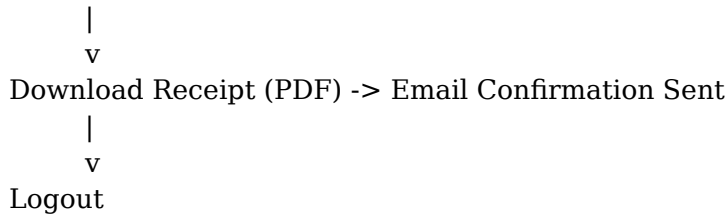


Figure 11.3 — Parent fee payment E2E flow: 6 steps across mobile UI + API + payment gateway + notification

11.6.4 Test Data for E2E

E2E tests run against the staging environment, which uses production-like masked data. Tests reference entities by stable natural keys (e.g., a specific parent's phone number) seeded into staging. Tests clean up after themselves — any entity created during the test (e.g., a new admission) is deleted in the teardown phase. Tests are designed to be idempotent — running twice produces the same result. Test data fixtures are versioned in git alongside the tests.

11.6.5 Failure Handling

When an E2E test fails, the framework captures: video recording of the run, browser/device trace, network request/response log, application logs, and a screenshot at the failure point. These artifacts are uploaded to the CI dashboard and linked in the failure notification (Slack + email). Engineers can reproduce the failure locally by running the same test against the same staging data. Flaky tests (intermittent failures) are quarantined after 3 flakes and require a fix before re-enabling.

11.6.6 E2E Suite Maintenance

E2E tests are the most expensive to maintain because UI changes can break many tests. PreOne mitigates this via the Page Object Model: every page has a corresponding Page class encapsulating its selectors + actions; tests interact with pages, not raw selectors. When a UI changes, only the Page class needs updating. The E2E suite is reviewed quarterly — tests that no longer reflect real user flows are removed; new critical flows are added. Suite size is capped at ~150 tests to keep execution under 30 minutes.

11.7 Performance Tests

Performance tests validate that the system meets latency, throughput, and scalability targets under realistic and peak loads. In a preschool SaaS, performance matters most at

peak moments — morning attendance marking (500+ teachers across 50 schools hitting the API simultaneously), fee payment deadlines (parents paying in the last 3 days of term), and report card generation (50+ principals generating term reports in the same hour). PreOne runs performance tests nightly on staging and on-demand for capacity planning. Performance regressions are treated as bugs, not optimizations.

11.7.1 Objectives

Performance tests serve four objectives. Each objective has a corresponding test type + metric. The objectives are not independent — a system that meets latency SLAs under load is also likely to scale, but each objective is validated explicitly because they fail in different ways.

Objective	Description
Validate scalability	System must handle 10x current peak traffic without degradation. Validated via load test ramped to 10x peak; metrics: latency, error rate, resource utilization.
Measure latency	Capture p50, p95, p99 latency per endpoint. Compare against SLA. Track latency trend over releases — regression > 10% triggers investigation.
Verify concurrency	1000 concurrent users must be served without errors. Database connection pool, Redis pool, K8s pod autoscaling validated.
Detect bottlenecks	Identify slow queries, N+1 patterns, lock contention, memory leaks, GC pauses. Profiling (CPU, memory, I/O) on every performance test run.

11.7.2 Test Types

Five performance-test types cover different load patterns. Each type validates a different failure mode. Tests are run on the cadence listed — nightly for load, quarterly for stress/spike/endurance/scalability. Results are tracked across releases to detect gradual regressions.

Test Type	Description
Load Test	Expected traffic (1x peak) sustained for 30 min. Validates normal-operation behavior. Baseline for regression detection. Run nightly on staging.
Stress Test	Beyond expected load (ramp to 5x peak). Identifies breaking point + failure mode. Run quarterly + pre-major-release. Results inform capacity planning.
Spike Test	Sudden traffic increase (0 → 3x peak in 30 sec). Validates autoscaling + queue buffering. Run quarterly. Common scenarios: result-day traffic, fee deadline, broadcast notification.
Endurance Test	Sustained expected load for 8 hours. Detects memory leaks, connection leaks, log-rotation issues. Run monthly + pre-release. Critical for long-running pods.
Scalability Test	Horizontal growth — add pods, measure throughput gain. Validates that doubling pods roughly doubles throughput (within 80% efficiency). Run quarterly + after architecture changes.

11.7.3 Target SLAs

Every API + UI flow has a performance SLA. SLAs are measured continuously via synthetic monitoring in production + validated nightly via k6 on staging. SLA violations trigger PagerDuty alerts. The table below lists the authoritative SLAs; these are the same SLAs communicated to enterprise customers in the MSA (Master Service Agreement).

Metric	Target	Scope	Tool
API Response (p95)	< 300 ms	Standard read/write endpoints under 1x peak load	k6 + Prometheus
API Response (p99)	< 800 ms	Standard read/write endpoints under 1x peak load	k6 + Prometheus

Metric	Target	Scope	Tool
Dashboard Load	< 2 sec	Initial page load + first contentful paint on web portal	Lighthouse CI
Login	< 1 sec	Submit credentials to JWT issued (excluding OTP delivery latency)	k6 + custom
Search	< 500 ms	Student/Staff search with 3+ character query, 10k+ records	k6 + Postgres EXPLAIN
File Upload	< 5 sec	10MB file via pre-signed URL (excluding client bandwidth)	k6 + S3 metrics
Report Generation	< 10 sec	Class report card PDF (30 students) generated server-side	k6 + BullMQ metrics
Mobile App Cold Start	< 3 sec	PreOne Hub cold start on mid-range Android device	Firebase Performance

11.7.4 Load Test Architecture

Load tests use k6, an open-source load-testing tool. k6 scripts are written in JavaScript and executed from a distributed cluster of load generators (running in a separate K8s namespace) to avoid the load generator itself becoming the bottleneck. Tests ramp up traffic over 5 minutes, hold peak for 15-30 minutes, then ramp down. Metrics (latency, throughput, error rate) are streamed to Prometheus + visualized in Grafana. Comparison reports against the previous run are auto-generated and posted to Slack.

11.7.5 Performance Regression Detection

Performance regressions are detected via comparison to baseline. Baseline is the p95 latency of each endpoint under 1x peak load, captured on the previous release. A regression > 10% triggers an investigation ticket. A regression > 25% blocks the release.

Regressions are often caused by N+1 queries, missing indexes, accidental full-table scans, or unbounded result sets. The Performance Engineer maintains a 'top 10 slow endpoints' report that is reviewed monthly.

11.7.6 Capacity Planning

Quarterly scalability tests inform capacity planning. The test doubles the load and measures how the system scales — ideally, throughput doubles with acceptable latency. If throughput does not scale linearly, the bottleneck is identified (CPU, memory, DB connections, Redis throughput, network) and a capacity-planning ticket is opened. The test also validates K8s horizontal pod autoscaler (HPA) behavior — pods should scale out within 60 seconds of load increase.

11.8 Security Tests

Security tests validate that the system resists attack. Unlike functional tests (which validate that the system does what it should), security tests validate that the system does not do what it should not — leak data, allow unauthorized access, execute injected code, or expose vulnerabilities. Security testing is layered: automated scans in CI (SAST, DAST, dependency, secret, image) catch common issues on every PR; manual penetration testing quarterly catches deeper issues. This chapter defines the security test suite; the authoritative reference for security controls is ADR-113.

11.8.1 Authentication Tests

Authentication tests validate the identity layer. Each test verifies that the system correctly accepts legitimate credentials and rejects illegitimate ones. Boundary cases (token expiry, OTP attempts) are explicitly tested because they are common attack vectors.

Test	Description
JWT Validation	Tampered signature → 401. Wrong algorithm (alg confusion attack) → 401. Missing required claims → 401. Valid JWT → passes.
Token Expiry	Access token expired (15 min) → 401 with TOKEN_EXPIRED code. Refresh token expired (30 days) → 401 with REFRESH_TOKEN_EXPIRED. Boundary: 1 second before/after expiry.

Test	Description
Refresh Token	Rotation on every use. Reuse of prior refresh token → entire session family revoked. Stolen-token detection validated. Token theft simulation in test.
OTP Validation	Correct OTP → success. Wrong OTP → 401 + attempt counter. 5 wrong attempts → account locked 15 min. OTP expiry (5 min) enforced. Resend cooldown (60s) enforced.
Session Timeout	Idle timeout (30 min) → token rejected. Absolute timeout (8 hours) → token rejected. Sliding session extended on activity up to absolute limit.

11.8.2 Authorization Tests

Authorization tests validate the access-control layer. Each test verifies that a user with a given role/permission can or cannot perform a specific action. The tests use a matrix approach: every role is tested against every endpoint, with expected allow/deny encoded as test data. This catches both missing-permission-check bugs and over-permissive bugs.

Test	Description
RBAC	Each role tested against each endpoint. Allow/deny matrix encoded in test data. Role escalation attempts (user modifies own roles via API) → 403.
Permission Checks	Granular permission (e.g., STUDENT.MEDICAL_READ) tested independently of role. User with permission → 200; without → field stripped or 403.
Scope Validation	TenantId in JWT must match requested resource. SchoolId/branchId scope enforced. Cross-scope access → 403 + audit log.
Tenant Isolation	Tenant A user cannot read/write Tenant B data. Tested at API + service + repository + DB level. SQL injection attempts to bypass tenant filter → blocked by Prisma parameterization.

Test	Description
Branch Isolation	Branch-scoped roles (Teacher) cannot access other branches within same tenant. Principal can access all branches in school. Validated per API endpoint.

11.8.3 OWASP Top 10 Coverage

PreOne's security test suite provides explicit coverage for every OWASP Top 10 (2021) risk. The table below lists each risk, the test that validates mitigation, and the tool used. Coverage is validated quarterly via external penetration testing and continuously via automated SAST/DAST in CI. Findings from any source feed the security backlog with severity-based SLAs.

OWASP Risk	Test	Tool
Broken Access Control	BOLA/IDOR tests — attempt to access other user's resources by ID manipulation. RBAC matrix tests. Privilege escalation tests.	Custom + OWASP ZAP
Injection (SQL/NoSQL)	SQLi payloads in every input field. NoSQL injection in query params. Command injection in file-name fields. Prisma parameterization validated.	SQLMap + OWASP ZAP
XSS	Reflected XSS in query params. Stored XSS in user-input fields (notes, comments). DOM XSS in client-side routing. CSP effectiveness validated.	OWASP ZAP + manual
CSRF	Cross-origin POST with forged token. SameSite cookie enforcement. JWT-in-header pattern (CSRF-immune) validated.	OWASP ZAP + custom

OWASP Risk	Test	Tool
Security Misconfiguration	Default credentials scan. Directory listing check. Verbose error messages. Debug endpoints exposed. CIS benchmark scan.	Nikto + CIS-CAT
Cryptographic Failures	TLS version + cipher suite scan. Certificate validity + chain. HSTS + HSTS preload. Weak crypto (MD5, SHA1, DES) in code scan.	SSL Labs + SAST
SSRF	User-supplied URL fields (webhook, avatar) tested with internal URLs (169.254.169.254, localhost, internal DNS). Egress firewall validated.	Custom + manual
Sensitive Data Exposure	PII in URL params. PII in logs. PII in error messages. Response payload scan for unmasked PII. Audit log access tests.	Custom + DAST

11.8.4 Security Activities

Beyond the per-test coverage above, PreOne runs continuous security activities listed below. Each activity has a tool, a cadence, and a defined remediation SLA. Activities are integrated into CI where possible (SAST, dependency, secret, image) and run as scheduled jobs or external engagements for the rest (DAST, pentest). The security dashboard tracks findings by severity + age.

Activity	Tool	Cadence	Description
SAST	SonarQube + Semgrep	Every PR + nightly main	Static analysis of source for injection, hardcoded secrets, weak crypto, insecure patterns. PR fails on High.

Activity	Tool	Cadence	Description
DAST	OWASP ZAP	Nightly on staging	Black-box scan of running app. Crawls routes, fuzzes inputs, detects OWASP Top 10. Findings triaged within 48h.
Dependency Scanning	Snyk + npm audit + Dependabot	Every PR + daily	CVE check on all dependencies. Auto-PR for patches. Block merge on Critical/High with fix available.
Secret Scanning	git-secrets + TruffleHog + GitHub Secret Scan	Pre-commit + PR + daily	Detects AWS keys, DB creds, tokens in git history. Blocks commit. Auto-rotates if leaked.
Container Image Scanning	Trivy + ECR scan-on-push	Every build	OS package CVEs + app dependency CVEs + misconfig in images. Blocks deploy on Critical with fix.
Penetration Testing	External vendor + internal	Quarterly + pre-major-release	Manual + automated pentest by certified team. Covers OWASP Top 10 + business logic + multi-tenant isolation. Findings fixed + re-tested.

11.8.5 Remediation SLAs

Security findings have severity-based remediation SLAs aligned with industry standards. Critical: 24 hours (block release if unfixed). High: 7 days. Medium: 30 days. Low: 90 days. SLAs are tracked on the security dashboard; overdue findings are escalated to engineering

leadership. False positives are suppressed with written justification + re-validation date. SLA breaches are reviewed monthly by the security steering committee.

11.9 Test Data Strategy

Test data is the foundation of every test — a test is only as reliable as the data it runs against. PreOne's test data strategy ensures that tests are deterministic, isolated, and realistic. The cardinal rule is: no production data in testing. Production data contains real PII of children, parents, and staff; copying it to lower environments would violate DPDP and betray customer trust. Instead, PreOne uses synthetic data (generated to mimic production patterns) and masked data (production-derived but with PII replaced) for environments that need realistic scale.

11.9.1 Principles

The principles below govern test data across all environments. Each principle is enforced via tooling or process; deviations require QA Lead approval. The goal is a test suite that produces the same result on every run, in every environment, without ever touching real customer data.

Principle	Description
Deterministic Test Data	Tests must produce the same result on every run. No reliance on current date (use injected clock), no reliance on external APIs (mocked), no reliance on test execution order.
Repeatable Seed Data	Seed scripts produce the same dataset every time. No random IDs without seed. No reliance on auto-increment values. Tests reference entities by stable natural keys.
Isolated Test Databases	Each test suite runs against its own database instance (Docker container). Tests do not share state. Database is reset (transaction rollback or full re-seed) between tests.
No Production Data in Testing	Production data never copied to lower environments. Contains real PII of children — DPDP violation. Synthetic data only in dev/QA/staging.

Principle	Description
Masked Data for Staging	When staging needs production-like volume, data is synthesized from production patterns (not copied) with names/phones/emails masked. Masking is irreversible; audited.
Version-Controlled Seed Scripts	Seed scripts live in git alongside code. Versioned per release. Migrations applied to seed DB on every PR. Seed scripts themselves are tested.

11.9.2 Environment-wise Data Strategy

Each environment uses a different data strategy appropriate to its purpose. Local and CI use minimal seed data for speed; QA uses a synthetic enterprise dataset for realistic scale; staging uses production-like masked data for production-simulation fidelity. The table below documents the strategy per environment.

Environment	Data	Description
Local	Seed Data	Minimal seed — 1 tenant, 1 school, 1 branch, 10 users (1 per role), 20 students, 50 parents. Refreshed via 'npm run db:seed'. Developer can extend locally.
CI	Fresh Database + Seed	Database created from scratch on every CI run. Migrations applied. Seed scripts run. Isolated per test suite. Torn down after run. No state leakage between builds.
QA	Synthetic Enterprise Dataset	Realistic scale: 5 tenants, 20 schools, 50 branches, 500 users, 5000 students. Generated by data-generation script. Refreshed nightly. QA engineers can mutate; reset overnight.

Environment	Data	Description
Staging	Production-like Masked Data	Production schema + production volume (anonymized). Names replaced with realistic fake names. Phones/emails/Aadhaar masked. Refreshed weekly from production snapshot (masked).
UAT	Subset of Staging	Subset of staging data scoped to UAT testers' tenant. Refreshed on demand. UAT testers use their own accounts; data is realistic but synthetic.
Production	Live Data Only	Real customer data. No test data, no seed scripts. Smoke tests use a dedicated test tenant (clearly marked) created by ops team. Monitored for abuse.

11.9.3 Standard Seed Dataset

The standard seed dataset is the canonical test data used in local dev + CI + QA. It is structured to exercise every role, every domain, and every relationship in the system without being overwhelming. The tree below illustrates the seed structure. The seed is generated by a script (`npm run db:seed`) that is itself tested — a broken seed script blocks CI just like a broken test.

```

Tenant (PreOne Demo Tenant)
|__ School (Sunrise Preschool)
|   |__ Branch (Koregaon Park, Pune)
|   |__ Branch (Banjara Hills, Hyderabad)
|   |
|   |__ Academic Year (2026-2027)
|       |__ Terms (Term 1, Term 2, Term 3)
|       |__ Holidays (regional calendar)
|       |
|       |__ Users (All Roles)

```

```

|   |__ Super Admin (1)
|   |__ School Owner (1)
|   |__ Principal (1 per branch)
|   |__ Teacher (5 per branch)
|   |__ Finance User (1)
|   |__ HR User (1)
|   |__ Parent (20)
|   |__ Student (20)
|
|__ Students (20 — varied: age, gender, program)
|__ Parents (20 — varied: 1 parent, 2 parents, guardian)
|__ Teachers (10 — varied: full-time, part-time, guest)
|__ Admissions (20 — varied: pending, approved, rejected)
|__ Attendance (last 30 days for each student)
|__ Fees (invoices + payments for current term)
|__ Inventory (50 items across 5 categories)
|__ Reports (last term report card for each student)

```

Figure 11.4 — Standard seed dataset: hierarchical structure spanning all roles, domains, and relationships

11.9.4 Synthetic Data Generation

For environments that need realistic scale (QA, staging), PreOne uses a synthetic data generator. The generator produces realistic-looking but fake data: Indian names (first + last, varied by region), Indian phone numbers (with valid prefixes but unused), fake emails (@example.test domain), fake addresses (real Indian cities, fake street names), realistic fee structures (varied by school type), realistic attendance patterns (with seasonal variation). The generator is parameterized by scale — a small dataset for QA, a large dataset for staging.

11.9.5 Masking Pipeline

Staging uses production-derived masked data for production-simulation fidelity. The masking pipeline: (1) Snapshot production PostgreSQL; (2) Restore to an isolated masking environment; (3) Run masking scripts that replace PII columns (name, phone, email, Aadhaar, address) with realistic fake values, preserving structure (e.g., phone remains a 10-digit Indian number); (4) Hash any unique identifiers that could enable re-identification; (5) Audit-log every masking run; (6) Restore the masked snapshot to staging. The masking pipeline is itself tested — a PII scan of staging must find zero real PII.

11.9.6 Test Data Refresh Cadence

Test data refreshes on a per-environment cadence. Local: on-demand by developer (npm run db:seed). CI: fresh per build. QA: nightly refresh from seed + synthetic generator.

Staging: weekly refresh from production snapshot (masked). UAT: on-demand by product team. Production: never (live data only). Refresh logs are archived for audit. The masking pipeline is run by the DevOps team and is a gated operation — only specific IAM roles can execute it.

11.10 Test Environments

PreOne maintains six test environments, each with a distinct purpose, data strategy, and uptime SLA. The environment pipeline flows from Local (developer-owned) through CI (ephemeral) to QA, UAT, Staging, and Production. Promotion between environments is gated by quality gates (Section 11.11) and human sign-off. The separation ensures that no single environment serves conflicting purposes — e.g., QA engineers can break things in QA without affecting UAT sign-off, and staging accurately simulates production because no one is mutating its data ad-hoc.

11.10.1 Environment Catalog

The table below enumerates each environment, its purpose, a description, and its uptime SLA. Environments are provisioned via Terraform + Helm; configuration is versioned in git. Environment drift is detected daily via a config-comparison job.

Environment	Purpose	Description
Local	Developer testing	Docker Compose stack: NestJS + Next.js + React Native (Expo) + PostgreSQL + Redis + MinIO. Hot-reload. Seeded with minimal dataset. Developer-owned; no SLA.
CI	Automated validation	Ephemeral K8s namespace per build. Fresh DB. Runs unit + integration + API + lint + security scans. Torn down after build. 15-min SLA per build.

Environment	Purpose	Description
QA	Functional testing	Persistent environment for QA engineers. Synthetic enterprise dataset. Refreshed nightly. Multiple QA engineers share; conflicts resolved via feature flags. 99% uptime SLA.
UAT	Business acceptance	Pre-production environment for product + business stakeholders. Realistic data. Sign-off gate for production release. 99.5% uptime SLA. Change-frozen during UAT cycles.
Staging	Production simulation	Production-identical infrastructure (smaller scale). Production-like masked data. Performance + endurance tests run here. Final smoke-test gate before production. 99.9% uptime SLA.
Production	Smoke & Monitoring	Live customer data. Smoke tests run every 5 min against /health + 3 critical user flows via a dedicated test tenant. Synthetic monitoring via New Relic. 99.95% uptime SLA.

11.10.2 Promotion Gates

Promotion from one environment to the next is gated by automated checks + human sign-off. The table below lists each promotion path, the gate, and a description. Gates cannot be bypassed by individual engineers; VP-level approval is required for emergency overrides.

From	To	Gate	Description
Local	CI	Code review approval + commit to feature branch	Engineer pushes branch; CI runs full test suite on PR. Two reviewer approvals required for merge to main.
CI	QA	All CI checks green + main branch build	Every main-branch build auto-deploys to QA. QA engineers pick up new features for functional testing.
QA	UAT	QA sign-off + release notes prepared	QA lead signs off that new features meet acceptance criteria. Product manager prepares release notes for UAT.
UAT	Staging	UAT sign-off + business approval	Product + business stakeholders sign off. Release candidate tagged. Deployed to staging for final perf + smoke tests.
Staging	Production	All staging tests green + change-advisory approval	Performance + smoke tests pass on staging. Change Advisory Board (CAB) approves production deploy. Deploy window: Tue-Thu 10am-2pm IST.

11.10.3 Environment Hygiene

Each environment has hygiene rules to keep it usable. (1) Local: developer-owned, no rules. (2) CI: ephemeral, no rules needed. (3) QA: nightly refresh; QA engineers can mutate data but should not break the schema; long-running test data is preserved across refreshes via a

'preserve' tag. (4) UAT: change-frozen during UAT cycles; only the release candidate under test is deployed. (5) Staging: only release candidates deployed; no ad-hoc feature branches; performance tests run here nightly. (6) Production: only CAB-approved deploys; smoke tests only.

11.10.4 Environment Monitoring

All non-local environments are monitored via the same observability stack as production (Prometheus + Grafana + Loki + Sentry). Staging + Production have full alerting; QA + UAT have informational dashboards only. Environment health is published on a status page accessible to all engineers. Environment downtime is tracked; SLA breaches are reviewed monthly.

11.11 CI/CD Quality Gates

Quality gates are automated checks in the CI/CD pipeline that block promotion if criteria are not met. They are the enforcement mechanism for the testing philosophy — principles without gates are aspirations; principles with gates are law. PreOne's gates are layered: PR-level gates block merge, build-level gates block deploy to QA, release-level gates block deploy to staging/production. Gates cannot be bypassed by individual engineers; emergency overrides require VP-level approval + security team notification.

11.11.1 Gate Catalog

The gates below are mandatory for every build. Each gate has a tool, a description, and a defined failure action. A build can be promoted only if all gates pass. The gates are encoded in the GitHub Actions workflow; changes to the workflow require DevOps Lead approval.

Gate	Description	Tool
Unit Tests pass	All unit tests pass with coverage $\geq$ 85%. Failures block merge. Coverage drop > 2% blocks merge (forces new code to be tested).	Jest
Integration Tests pass	All service + repository integration tests pass. Database migrations apply cleanly. Test DB torn down + recreated per suite.	Jest + Testcontainers

Gate	Description	Tool
API Tests pass	All API tests pass against ephemeral API instance. OpenAPI spec validated against actual responses. Schema drift fails the build.	Supertest + Jest
E2E Tests pass	Critical E2E flows pass against staging deploy. Runs nightly; pre-release runs full suite. Failures block release but not PR merge (long-running).	Playwright + Detox
Security Scans pass	SAST + dependency scan + secret scan + container image scan all green. Critical/High with fix available → block. Critical/High without fix → security-team review + remediation SLA tracked.	SonarQube + Snyk + Trivy + git-secrets
Performance Benchmarks meet SLA	Critical endpoints benchmarked against SLA (p95 < 300ms). Regression > 10% fails the build. Tracked trend across releases.	k6 + Lighthouse CI
Code Coverage ≥ 85%	Overall line + branch coverage. Per-layer minimums: domain 95%, utility 100%, critical business logic 100%. Reported on every PR.	Jest coverage
No Critical or High vulnerabilities	Dependency + container + SAST scan results. Critical/High with available fix → block. Without fix → documented + remediation plan + security-team approval required.	Snyk + Trivy + SonarQube

Gate	Description	Tool
Database migrations validated	Migration applies cleanly on fresh DB + on production-snapshot copy. Rollback migration tested. Schema diff reviewed by DBA.	Prisma Migrate + custom
Linting and type checks pass	ESLint + Prettier + TypeScript strict mode. Zero errors. Warnings allowed but tracked. Code style consistent across team.	ESLint + Prettier + tsc

11.11.2 Enforcement Levels

Gates are enforced at three levels, corresponding to the promotion points in the environment pipeline. The table below lists each level, the gates enforced at that level, and a description. A failure at any level holds the build; the on-call engineer investigates within 30 minutes.

Level	Gates	Description
PR-level (blocks merge)	Unit + Integration + API + Security + Coverage + Lint	Every PR must pass these gates before merge. Two reviewer approvals also required. Bypass requires VP-level approval + security-team notification.
Build-level (blocks deploy to QA)	E2E smoke (10 critical flows) + Image scan + Migration validation	Every main-branch build must pass before auto-deploy to QA. Failure holds the build; on-call engineer investigates within 30 min.
Release-level (blocks deploy to Staging/Prod)	Full E2E suite + Performance benchmarks + DAST scan + Pen-test sign-off (quarterly)	Every release candidate must pass. CAB reviews gate results before production approval. Release manager owns the gate.

11.11.3 Gate Bypass Policy

Gate bypass is a high-risk operation and is restricted. The policy: (1) PR-level gates: no bypass; the gate must pass. (2) Build-level gates: on-call engineer can bypass with Slack acknowledgment + ticket for follow-up. (3) Release-level gates: VP-level approval required + security team notification + post-release follow-up ticket. Bypasses are audited and reviewed in the next sprint retrospective. The goal is zero bypasses per quarter; sustained bypasses trigger a process review.

11.11.4 Gate Reporting

Every build generates a Quality Gate Report published to the CI dashboard + engineering Slack. The report includes: gate name, pass/fail status, duration, link to logs, link to coverage report, link to security findings. A weekly summary aggregates gate-pass rates, average build duration, and top failure reasons. The summary is reviewed in the engineering all-hands monthly. Gate metrics feed the engineering OKR process.

11.11.5 Gate Maintenance

Gates themselves require maintenance. Each gate has a named owner accountable for: (1) keeping the gate's tooling current, (2) tuning thresholds to minimize false positives, (3) reviewing gate-failure trends, (4) proposing new gates as new risk classes emerge. The QA Lead owns the gate catalog as a whole; quarterly review adds, removes, or tunes gates based on effectiveness. A gate that has not caught a real defect in 6 months is reviewed for removal.

12. Deliverables

This Test Strategy produces the deliverables below. Each deliverable has a named owner and is a living artifact — updated as the system evolves. The deliverables are interlinked: the Test Case Catalog references the Test Strategy for principles; the API Test Collection references the OpenAPI Spec for contracts; the Coverage Report references the Test Case Catalog for traceability. Together, they form the PreOne testing knowledge base.

Deliverable	Description	Owner
Test Strategy Document	This document. Authoritative reference for testing approach, principles, pyramid, environments, gates. Owned by QA Lead; reviewed annually.	QA Lead
Test Plan	Per-release test plan: scope, schedule, resources, risks, entry/exit criteria. Derived from this strategy. Approved by Product + Engineering + QA.	Release Manager
Test Case Catalog	Comprehensive catalog of test cases organized by module. Each case has ID, description, steps, expected result, traceability to PRD/BRC. Living document in TestRail.	QA Engineers
Test Data Catalog	Inventory of seed scripts, data-generation scripts, masking scripts, and synthetic datasets. Versioned in git. Indexed by environment + use case.	QA Engineering
API Test Collection	Postman + Supertest collection covering all 530 APIs. Positive + negative + boundary + auth + validation tests. Synced with OpenAPI spec; drift fails CI.	QA + Backend
E2E Automation Suite	Playwright (web) + Detox (mobile) + Supertest (API) suites covering 10 critical user flows. Runs nightly + pre-release. Video + trace on failure.	QA Automation

Deliverable	Description	Owner
Performance Test Suite	k6 scripts for load + stress + spike + endurance + scalability. Lighthouse CI for web perf. Run nightly on staging + on-demand for capacity planning.	Performance Engineer
Security Test Checklist	OWASP Top 10 coverage matrix + SAST/DAST/dependency/secret/image scan results + pentest findings register. Reviewed quarterly + after incidents.	Security Engineer
CI/CD Quality Gate Report	Per-build report: gate status, coverage, vulnerabilities, perf benchmarks. Published to engineering Slack + archived in CI dashboard.	DevOps Engineer
Coverage Report	Per-build coverage report: overall + per-layer + per-module. Trend over time. Coverage drop alerts. Published to engineering dashboard.	QA Engineering

12.1 Deliverable Lifecycle

Each deliverable follows a lifecycle: draft → review → published → maintained → retired. Drafts are works-in-progress, not yet authoritative. Reviewed deliverables have been peer-reviewed + QA-Lead-approved. Published deliverables are authoritative and linked from the engineering wiki. Maintained deliverables are updated as the system evolves. Retired deliverables are archived but kept for historical reference. The Test Strategy Document itself is published + maintained; this version (1.0) is the initial published version.

12.2 Deliverable Storage

Deliverables live in distributed locations appropriate to their format. The Test Strategy + Test Plan + Test Case Catalog live in Confluence (versioned, searchable, linkable). The API Test Collection + E2E Automation Suite + Performance Test Suite live in git alongside code (versioned, reviewable, executable). The CI/CD Quality Gate Report + Coverage Report are auto-generated and live in the CI dashboard (no manual maintenance). The Security Test Checklist lives in the security team's GRC tool. All locations are linked from the engineering wiki.

13. Cross References

This Test Strategy does not stand alone — it is part of the PreOne architecture baseline. The references below link this document to predecessor and successor documents. Engineers implementing tests should consult these references for full context. In particular, BRC v1.0 is the source of business rules that must be tested, and the OpenAPI Spec is the source of API contracts that API tests validate.

Reference	Topic	Description
Vision v1.0	Product vision + quality goals	Quality + reliability goals that this Test Strategy operationalizes.
BRC v1.0	Business rules + compliance + audit	Source of business rules that must be tested. Every BRC rule has at least one test case with traceability tag.
Master PRD v1.0	Functional + non-functional requirements	Requirements traceability matrix maps every PRD requirement to test case(s).
DDD v1.0	Bounded contexts + domain events	Defines service boundaries that integration tests validate; domain events that event-handler tests cover.
ERD v3.0	Database schema + constraints	Schema that repository integration tests validate. Constraints (FK, unique, check) tested via negative cases.

Reference	Topic	Description
API Contract Catalog v1.0 + OpenAPI Spec v1.0	API contracts + security schemes	Authoritative source for API test cases. Drift between spec + implementation fails CI.
ADR-111 DevOps v1.0	CI/CD pipeline + environments	Pipeline that runs the quality gates; environments described in Section 11.10.
ADR-113 Security v1.0	Security architecture + controls	Security controls validated by Section 11.8. SAST/DAST/pentest findings feed the security backlog.

13.1 Document Control

Document ID	Test Strategy v1.0 (Chapter 11)
Title	PreOne Enterprise Test Strategy
Version	1.0
Status	Test Strategy Freeze
Date	2026-07-14
Owner	PreOne QA & Engineering Team
Review Cadence	Quarterly + after major incidents
Next Review	2026-10-14
Change Control	Amendment requires QA Lead + Engineering Lead approval
Distribution	Internal Engineering — Confidential

14. Glossary

The glossary below defines testing + QA terms used throughout this document. Terms are listed alphabetically. Where a term has multiple meanings in industry usage, the PreOne-specific meaning is given.

Term	Definition
Test Pyramid	Layered testing model: many fast unit tests at the base, fewer slower integration/API tests in the middle, very few slowest E2E tests at the top.
Shift Left	Practice of moving testing earlier in the development lifecycle — requirements review, test planning, and unit testing before code completion.
Smoke Test	Quick test that verifies the most critical functions work. Used after deploy to catch environment-level breakage before users hit it.
Regression Test	Test added after a bug fix to ensure the bug never reappears. Every bug fix requires a regression test before merge.
BOLA / IDOR	Broken Object Level Authorization / Insecure Direct Object Reference — vulnerability where a user can access another user's objects by manipulating IDs.
SAST / DAST	Static / Dynamic Application Security Testing — source-code vs. running-app security scanning.
p95 / p99	95th / 99th percentile latency — 95% / 99% of requests complete within this time. More meaningful than average for performance SLAs.
Endurance Test	Long-running load test (typically 8+ hours) to detect memory leaks, connection leaks, and gradual degradation.
Idempotency Key	Client-provided unique key on POST requests. Server caches response; replay returns cached result. Prevents duplicate side effects on network retry.
Test Container	Docker container spun up per test suite for isolated testing of integrations (PostgreSQL, Redis, MinIO). Torn down after suite.

Term	Definition
Traceability Matrix	Mapping of requirements (PRD/BRC) to test cases. Auto-generated from test metadata. Surfaces untested requirements.
Quality Gate	Automated check in CI/CD that blocks promotion if criteria not met. Cannot be bypassed by individual engineers.
Synthetic Data	Artificially generated data that mimics production data structure + volume without containing real PII. Used in QA + staging.
Masked Data	Production data with PII fields replaced/masked. Used in staging for realistic volume. Masking is irreversible + audited.
CAB	Change Advisory Board — cross-functional team that reviews + approves production deployments. Meets daily during release windows.